

# 从已有代码仓开始

## Coding Agent 研发最小闭环

数据库部 AI 辅助研发实战培训 · L01



# 为什么从“已有代码仓”开始

第一幕 | 为什么重要

真实研发里，大多数工作不是从零写项目，而是在已有系统里修 bug、补测试、改小功能。

## 从零写项目

上下文少，约束少  
适合快速原型  
失败成本相对较低

## 已有代码仓

上下文多，隐含约束多  
容易改错地方  
失败可能破坏已有逻辑

## Agent 常见失败

不知道项目上下文  
不知道修改边界  
不知道如何验证

AI 辅助研发失败，很多时候不是模型不会写代码，而是任务边界、上下文和验证方式没给清楚。

# Agent 心智模型

心智模型与闭环概览

把 Agent 当作一位刚入职、能力很强但完全不了解你们代码的同事。

## 你不会这样做

第一天就让新同事直接改核心代码。  
只说“你修一下”  
改完就相信他说“改好了”

## 你会这样做

先读README、目录和关键模块  
先说背景，目标，约束，验收  
跑测试，看diff,写总结

## 角色分工

你是 driver，agent 是 copilot。  
你：定义目标，控制边界，审核计划，决定何时执行  
Agent: 快速读写代码, 执行重复任务, 生成候选方案, 帮助定位失败

# 最小研发闭环

心智模型与闭环概览



## 每阶段都有输入

任务说明、相关文件、约束、测试输出。

## 每阶段都有输出

理解摘要、修复计划、diff、测试结果、总结。

## 每阶段都有失败信号

泛泛描述、超范围 diff、重复修错、绕过测试。

# 实战项目：SQLite-backed KVStore

第二幕 | 先看翻车

```
exercise/
├── src/kvstore/
│   ├── store.py          ← SQLite KV 引擎, 主要 bug 在这里
│   ├── persistence.py    ← JSON snapshot, bug 也在这里
│   ├── parser.py         ← 命令解析, 不要动
│   └── cli.py            ← REPL, 不要动
├── tests/                ← 5 条 baseline 失败测试
└── validators/validate.sh
```

```
CREATE TABLE kv (
  key      TEXT PRIMARY KEY,
  value    TEXT NOT NULL,
  expires_at REAL
)
```

## 过期语义

expires\_at IS NULL: 没有 TTL

expires\_at > now: 尚未过期

本课 bug 都和“过期 key 的可见性”有关

练习背景：3 个同源 bug 导致 5 条 baseline 测试失败。

## 错误示范：一句话让 AI 直接修

先跑：  
./validators/validate.sh

坏 prompt：  
“帮我修这 5 个测试”

## 这条 prompt 缺了什么？

没有背景：Agent 不知道 KVStore 的整体机制

没有上下文：Agent 不知道该读哪些文件

没有目标：Agent 不知道到底要修哪类行为

没有约束：Agent 不知道哪些文件不能动

没有验收：Agent 不知道怎么证明修好了

示例：keys()和\_\_len\_\_修复正常，但contain却只在python层做后过滤，并引入了我们没要求的evict操作

```
def __contains__(self, key: str) -> bool:
-     return (
-         self._conn.execute(
-             "SELECT 1 FROM kv WHERE key = ?", (key,)
-         ).fetchone()
-         is not None
-     )
+     now = time.time()
+     row = self._conn.execute(
+         "SELECT expires_at FROM kv WHERE key = ?", (key,)
+     ).fetchone()
+     if row is None:
+         return False
+     expires_at = row[0]
+     if expires_at is not None and now >= expires_at:
+         self._evict(key)
+         return False
+     return True

def _evict(self, key: str) -> None:
    self._conn.execute("DELETE FROM kv WHERE key = ?", (key,))
```

为什么会这样写

大概率是它看到了 delete() 和 expire() 的 Python 层判断模式（先 SELECT，再判断 expires\_at，过期则 evict），然后对 \_\_contains\_\_ 也套用了同样的模式。

这说明它没有区分两种场景：

- delete()/expire() 需要 Python 层判断，因为它们有写操作语义（delete 要决定返回 True/False，expire 要更新 TTL）
- keys()/\_\_len\_\_/\_\_contains\_\_ 是纯查询，应该在 SQL 层就过滤干净

它没有识别出“纯查询方法应该统一用 SQL 过滤”这个设计意图，而是机械地复制了最近看到的代码模式。

# 翻车后果分析

现场演示

| 可能的翻车形态                         | 根因                    | 正确做法                       |
|---------------------------------|-----------------------|----------------------------|
| 一次改很多文件                         | 没有要求先计划，分步验证，没有列文件白名单 | 列出白名单，要求先给计划，多个小修改比一次大修改要好 |
| 改 schema 加列                     | 没写不允许改 schema /缺少约束   | 写进【约束】段                    |
| Python 层后过滤                     | agent 求稳走捷径 /缺少任务目标   | 引导回 SQL 层                  |
| 改已有测试绕过                         | 没明确不能改已有测试 /缺约束       | 红线写进 prompt                |
| Agent 优先处理理想路径，忽略边界情况，异常情况和性能隐患 | Agent 倾向于先处理理想路径      | 不要指望Agent能自动想到边界情况，要显式的提到  |

原则：测试红了不可怕；“为了绿而破坏契约”才可怕。

# Prompt怎么写：任务描述模板

任务描述

**【背景】** 这段代码做什么、为什么要改它（1—3 句）

**【目标】** 这次改动要达成的可验证结果（1 句）

**【约束】** 哪些文件能改 / 哪些不能改 / 不改的行为

**【验收】** 跑哪条命令、看到什么算成功

## 关键技巧

能用命令验收，就不要只用文字描述

目标只写一个，不要混多个需求

约束要写“不要做什么”

上下文很重要：告诉 Agent 读哪里，再告诉它怎么改



## 反例 vs 正例对照

### ✗ 反例

“GET 一个过期的 key 应该返回 None，看下哪里不对”

问题：

- 没说项目结构
- 没说可改文件
- 没说验收命令
- 没说禁止改测试 / schema / API

结果：agent 容易自由发挥，diff 失控。

**左边：Agent 自由发挥，可能 200 行 diff**

### 【背景】

src/kvstore/store.py 是 SQLite-backed KV 引擎。  
kv 表包含 key, value, expires\_at。  
get() 已经在 Python 层做了懒过期判断。

### 【上下文】

请阅读 store.py，重点看每个方法使用的 SQL。  
请阅读 persistence.py。  
请阅读 tests/test\_store.py 中和过期 key 相关的测试。

### 【目标】

让 keys() / \_\_len\_\_ / \_\_contains\_\_ 的 SQL 过滤已过期 key，  
与 get() 行为一致。

### 【约束】

只允许修改 store.py。  
不改 schema，不改 get()/ttl()，不改公开 API 签名，不改已有测试。

### 【验收】

./validators/validate.sh 通过。

**右边：精准改 3 条 SQL，能验证**

# Prompt怎么写：定范围约束

限制修改范围

| 层级   | 说明                         | 本课示例                                   |
|------|----------------------------|----------------------------------------|
| 文件范围 | 明确只允许修改哪些文件                | store.py + persistence.py              |
| 接口边界 | 不改签名、不改 schema、不加依赖        | 不改 get()/ttl()、不改公开 API                |
| 行为边界 | 已有输出格式和语义保持一致，有哪些已有的行为必须保持 | CLI / parser 不变；JSON schema 不变；已有的测试不变 |

约束：

- 只允许修改：src/kvstore/store.py、src/kvstore/persistence.py
- 只允许新增：tests/ 中的新测试函数
- 禁止：修改任何已有测试；修改 parser.py 或 cli.py；新增第三方依赖；修改 kvStore 公开方法签名
- 风格：保持现有 type hint 风格；不引入新抽象层

# 项目级配置约束：AGENTS.md

限制修改范围

## 是什么

项目根目录的 Markdown 文件，给 agent 的“项目须知”。

## 为什么重要

命令、风格、红线、权限边界，一进仓就告诉 agent。没有它，每次从零理解；有了它，进来就知道规矩。

## 维护建议

随代码提交 Git；流程变了同步更新。过时指令比没有指令更糟。

```
# AGENTS.md
## Project
SQLite-backed KVStore — 教学用微型 KV 引擎，Python 3.10+，零第三方依赖。

## Commands
- Test all: python3 -m unittest discover -s tests -v
- Validate: ./validators/validate.sh

## Boundaries
- Do NOT modify kv table schema
- Do NOT modify get() / ttl() implementations
- Do NOT add third-party dependencies
```

# AGENTS.md: 写什么 vs 不写什么

限制修改范围

✔ 应该写

Agent 猜不到的命令（测试 / 构建 / 验证）

与默认不同的代码风格规则

项目特有的架构决策和约束

常见陷阱和非显而易见的行为

红线：哪些操作绝对不能做

✘ 不用写

Agent 读代码就能推断的东西

语言本身的标准约定

详细 API 文档（给链接即可）

经常变化的信息

逐文件的代码库描述

臃肿的 AGENTS.md 反而有害：太长会导致 Agent 忽略真正重要的指令。

关键判断：每写一行，问自己“删掉这行 Agent 会犯错吗？”——如果不会，就删掉。

# Hooks — 把建议变成保证

系统强制，Agent 无法跳过

⚠️ **Soft Constraint (建议)**

写在 AGENTS.md

“跑完测试再提交”

Agent 可能忽略

context 满了更容易遗忘

类比：交通规则

🔒 **Hard Constraint (保证)**

配置为 Hook / Plugin

编辑后自动跑测试

系统强制执行

不受 context window 影响

类比：安全带

## Hook workflow



## OpenCode 原生：JS/TS Plugin

.opencode/plugins/auto-format.ts

```
export const AutoFormat = async ({ $ }) => ({
  "tool.execute.after": async (event) => {
    if (event.tool === "Edit")
      await `${ruff format ${event.input.file_path}`
  }
})
```

.opencode/plugins/ 目录下放 TS/JS 文件，OpenCode 自动加载

## 社区方案：YAML Hooks

.opencode/hook/hooks.yaml

```
hooks:
- event: tool.after.Edit
  run: "ruff format $FILE_PATH"

- event: tool.before.Bash
  run: "block DROP TABLE / rm -rf"
  on_fail: exit 2 # 拦截
```

### 常用 Hook 场景

| 场景         | 事件               | 动作          |
|------------|------------------|-------------|
| 编辑后自动格式化   | tool.after.Edit  | 跑 formatter |
| 拦截危险命令     | tool.before.Bash | exit 2 阻止   |
| 编辑后跑测试     | tool.after.Write | 跑 unittest  |
| 文件变化时 lint | file.edited      | 跑 linter    |

核心判断：

绝对不能违反 → 用 Hook（硬约束） · 偶尔可以例外 → 写 AGENTS.md（软约束） · 两者配合 = Agent 既灵活又安全

## 先读后改

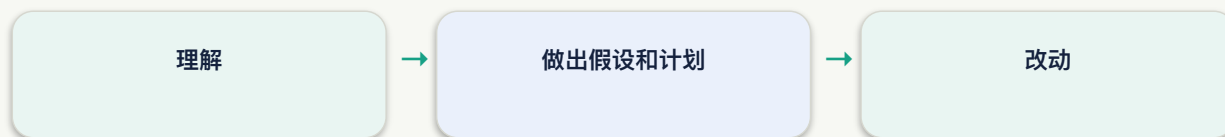
先读后改

在动手之前，先做以下三件事，只输出文字，不要改任何代码：

1. 阅读 `src/kvstore/store.py` 与 `tests/`，用 5 句话描述过期机制怎么工作。
2. 给出对 `task.md` 中现象的至少 1 个具体假设，并指出怀疑的代码行。
3. 给出修改计划：要动哪个方法、加几行、怎么测。

我会 review 你的输出，确认后再让你动代码。

本课专属：请先把每个方法用的 SQL 列出来，逐条标注是否漏了 `WHERE` 过滤 `expires_at`。



先让 Agent 暴露它的理解，再决定要不要让它动手。

# Plan Mode

Shift+Tab 切换 · Agent 物理上不能编辑文件 · 计划就绪后选择执行模式

## ① 进入 Plan Mode

`Tab` 或 `--permission-mode plan`

Agent 只能：读文件、跑探索命令、输出计划  
Agent 不能：编辑任何文件（系统强制）



## ② 审计划 — 四件事

- 文件清单只含授权文件？
- 有没有冒出新 class / helper / 文件？
- 每个改动点预估多大？（几行 vs 几十行）
- 改动顺序支持分步验证？



## ③ 批准 → 选执行模式

- ▶ Auto Mode 全自动，后台安全检查
- ▶ Accept Edits 自动改文件，命令要审批
- ▶ 手动审批 逐条确认每个操作
- ▶ 继续 planning 给反馈，不批准

### ✗ Prompt 里说“先别改”

- 靠 Agent 自觉
- 可能被忽略
- 没有正式审批流程
- 不知道计划何时“结束”

### ✓ Plan Mode

- 系统层面禁止编辑
- 物理上不可能改文件
- 批准后选执行模式
- Agent 主动提交计划

### 何时该进 Plan Mode?

- 修改 ≥ 2 个文件
- 对修改方向不确定
- 不熟悉要改的代码区域

### 何时不用?

- 改动一句话能描述清楚  
（修 typo、加 log、改变变量名）

# Agent 真的读了代码吗

先读后改

| Agent 输出                                               | 判断                 |
|--------------------------------------------------------|--------------------|
| "store.py 的 SELECT key FROM kv 缺少 WHERE expires_at 过滤" | ✅ 真读了：有函数名和 SQL 片段 |
| "建议检查过期逻辑是否完善"                                         | ❌ 在猜：教科书式泛泛描述      |
| "get() 判断 time.time() >= expires_at 后调用 _evict()"      | ✅ 真读了：引用具体实现细节     |
| "可能需要优化数据库查询"                                          | ❌ 在猜：没有代码引用        |

如果 agent 说不清楚，说明它还没读懂，不要让它继续改。



## 跑测试与修失败

```
./validators/validate.sh  
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

测试红了，第一件事不是改代码：

“这条测试为什么失败？是测试期望错了，还是实现错了？  
给我两种可能性的论据，再告诉我你倾向哪一种。”

### 何时该停下

- 同一处反复修了  $\geq 3$  次还没绿
- diff 越来越大：10 行  $\rightarrow$  100 行
- Agent 开始吞异常让测试过
- Agent 修改了明确禁止的文件

停下后：复盘 prompt  $\rightarrow$  收紧约束  $\rightarrow$  重新派活。

# 停下后的具体操作：不要在污染的 context 里硬拧

现场演示

关键规则：同一问题纠正超过 2 次还没好 → 不要继续纠正 → /clear + 更好的 prompt 重来。

| 操作     | 快捷键 / 命令                                         | 作用                     |
|--------|--------------------------------------------------|------------------------|
| 立刻停止   | Esc                                              | 中断执行，保留上下文，可以重定向       |
| 回到好的状态 | 让agent用 Git checkpoint (commit / stash / branch) | 恢复代码 + 对话到之前的检查点       |
| 撤销改动   | 告诉 Agent “Undo that”                             | 回退刚才的改动                |
| 完全重来   | /clear                                           | 清空上下文，用更好的 prompt 重新开始 |

# Context Window 管理：会话越长 Agent 越差

限制修改范围

| 场景           | 操作                   | 说明                         |
|--------------|----------------------|----------------------------|
| 切换到不相关任务     | /clear               | 清空上下文，避免“厨房水池式会话”          |
| 长会话中上下文膨胀    | /compact             | 压缩历史，但保留关键信息               |
| Agent 改错了想回退 | 告诉 Agent “Undo that” | 回到之前的检查点，恢复代码 + 对话         |
| 需要大量读文件做调查   | 用 subagent           | 调查在独立 context 中进行，主会话只接收总结 |

与本课的关系：分步修 Bug（Bug 1 → Bug 2 → Bug 3）时，每组修完可以 /clear 再进入下一组。  
常见反模式：“厨房水池式会话”——一个会话里同时修 bug、问架构、查文档，context 被不相关信息塞满。

## validation\_report.md

写总结与产出物

| 段落              | 写什么             | 示例                                     |
|-----------------|-----------------|----------------------------------------|
| Initial Failure | 初始失败命令与摘要       | validate.sh 输出 5 FAILED                |
| Root Cause      | 根因，引用具体 SQL 字符串 | SELECT key FROM kv 缺少 WHERE expires_at |
| Changes         | 改了哪些文件、哪段逻辑     | store.py 4 个方法 + persistence.py dump() |
| Validation      | 最终验证命令与结果       | [OK] L01 KVStore validation passed     |
| Remaining Risks | 未覆盖场景或潜在风险      | expires_at == time.time() 边界           |

基于本次会话的所有改动，按 validation\_report.template.md 的五段结构生成草稿，每段不超过 5 行，不夸张表达功能。

# context.md

写总结与产出物

非行业标准，但可用于复盘

| 板块               | 内容                 | 训练价值              |
|------------------|--------------------|-------------------|
| 提供给 Agent 的材料清单  | 列出具体喂了哪些文件         | 理解“上下文即质量”        |
| 刻意没提供的内容         | 例如没给 cli.py、没给全部测试 | 比“给了什么”更有训练价值     |
| 范围约束             | 哪些文件能改、哪些不能        | 和 prompt 的【约束】段呼应 |
| 验证方式             | 用了什么命令验证           | 闭环意识              |
| Agent 的错误判断与人工纠正 | 至少记录一次 agent 错了你纠了 | 这是最核心的学习产出        |

# 总结：最小闭环的 3 个核心信念

总结与练习

AI 辅助研发的核心能力，不是“会问”，而是会组织一次可控交付。

1

## 上下文即质量

Agent 只能基于你给出的文件、测试、约束和背景工作。上下文越精准，输出越稳定。

2

## 范围决定风险

文件白名单、接口边界、行为不变量越清楚，diff 越可控，返工越少。

3

## 验证才算闭环

没跑验证不算完成；没写总结不算闭环。最终要能说明根因、改动和风险。

每次让 Agent 改已有代码仓，都先问：上下文给够了吗？范围锁住了吗？验证跑通了吗？

课后练习交付：代码修改 + validation\_report.md + context.md

**Back Up**

# Backup A | 完整 Prompt 模板速查卡

Backup

五段式任务描述 ===

【背景】这段代码做什么、为什么要改它

【上下文】读哪些文件、重点看什么

【目标】可验证的结果，只写一个

【约束】白名单 + 红线，用三层限制法

【验收】跑什么命令、看到什么

=== 三步法：先读后改 ===

先做以下三件事，只输出文字，不要改代码：

1. 用 5 句话描述当前实现
  2. 给出至少 1 个具体假设，指出怀疑的代码行
  3. 给出修改计划：动哪个方法、加几行、怎么测
- 我会 review，确认后再让你动代码。

=== SQL 审查，本课专属 ===

请把每个方法用的 SQL 列出来，逐条标注是否漏了 WHERE 过滤。

=== 修失败 ===

这条测试为什么失败？测试期望错了还是实现错了？

给两种可能性的论据，告诉我你倾向哪一种。

=== 写总结 ===

按 validation\_report.template.md 五段结构生成草稿，每段不超过 5 行。



# Backup B | Plan Mode 详细操作

Backup

| 阶段           | 做什么                                | 重点              |
|--------------|------------------------------------|-----------------|
| 进入 Plan Mode | Shift+Tab 或 --permission-mode plan | 系统层面禁止编辑文件      |
| 只读探索         | 读文件、跑探索命令、输出计划                     | 不允许改代码          |
| 审计划          | 文件范围 / 多余抽象 / 改动规模 / 分步验证          | 不满意就继续 planning |
| 批准执行         | 选择 Auto / Accept Edits / 手动审批      | Ctrl+G 可直接编辑计划  |

## 核心判断

Plan Mode 不是更礼貌的 prompt，而是权限控制。  
适合：修改多个文件、方向不确定、不熟悉代码区域。

## Backup C | AGENTS.md 示例

Backup

```
AGENTS.md

## Project
SQLite-backed KVStore - 教学用微型 KV 引擎, Python 3.10+, 零第三方依赖。

## Commands
- Test all: `python3 -m unittest discover -s tests -v`
- Test single: `python3 -m unittest tests.test_store -v`
- Validate: `./validators/validate.sh`

## Conventions
- Type hints required, Google-style docstring
- SQL 用参数化查询 (? 占位符), 禁止字符串拼接

## Boundaries
- Do NOT modify kv table schema
- Do NOT modify get() / ttl() implementations
- Do NOT add third-party dependencies
```

# Backup D | Fowler 五个 AI 协作模式

Backup

| Fowler 模式                  | 含义              | 本课对应                              |
|----------------------------|-----------------|-----------------------------------|
| Knowledge Priming          | 给 AI 精选上下文      | 五段式【上下文】                          |
| Design-First Collaboration | 先设计 / 计划，再编码    | Plan Mode + 先读后改                  |
| Encoding Team Standards    | 把团队规范写成 AI 可读配置 | AGENTS.md                         |
| Context Anchoring          | 锚定工作范围          | 三层限制法                             |
| Feedback Flywheel          | 把协作经验沉淀回团队知识    | context.md + validation_report.md |

## Backup E | 测试选择的启示

Backup

### 不要泛泛说

“去跑测试。”

### 更好的说法

请先运行 tests/test\_store.py 中 StoreExpirationViewTest，确认过期 key 的可见性行为。

然后再运行 ./validators/validate.sh。

### 启示

指定测试文件，比泛泛要求“用 TDD”更有效

Agent 需要具体的验证路径

测试也是上下文的一部分

# Backup F | Hooks：把建议变成保证

Backup

| 类型              | 机制                      | 示例                          |
|-----------------|-------------------------|-----------------------------|
| Soft Constraint | AGENTS.md / prompt 里写规则 | “提交前跑测试”                    |
| Hard Constraint | Hook 自动执行或拦截            | 每次编辑后自动跑 formatter / linter |

| Hook 类型                | 触发时机        | 用途                     |
|------------------------|-------------|------------------------|
| PostToolUse Edit/Write | Agent 编辑后   | 自动跑 formatter / linter |
| PreToolUse Bash        | Agent 执行命令前 | 拦截危险命令                 |
| PostToolUse Bash       | Agent 执行命令后 | 自动跑相关测试                |

如果某个规则绝对不能违反，用 Hook；如果只是偏好或建议，写 AGENTS.md。

| 技巧                    | 场景               | 做法                   |
|-----------------------|------------------|----------------------|
| Subagent 调查           | 需要读大量文件，但不想污染主会话 | 让 subagent 调查后只返回总结  |
| Writer / Reviewer 双会话 | 修完代码后做客观 review  | 一个会话写，一个干净会话审        |
| Interview 模式          | 需求不明确，五段式写不出来    | 让 Agent 先采访你，再写 spec |

使用原则

这些是进阶技巧，不是 L01 的主线。  
先把“可控闭环”练熟，再逐步引入高级模式。

# 延伸资源

备用页

## OpenCode 文档

<https://opencode.ai/docs/>  
开源终端 AI coding agent, 支持终端 / 桌面 / IDE 入口。

## AGENTS.md 规范

<https://agents.md/>  
给 coding agent 的 README: 项目命令、风格、边界、测试约定。

## 本课代码仓

<https://github.com/XitongLiang/Lecture001>  
L01 正式版课程材料与 exercise。

## 下节课预告

从“修 bug”扩展到“小功能开发”: 需求拆解、测试先行与跨文件协作。

建议把本页链接放进课后材料, 并让学员直接从仓库 README 开始练习。